

Enterprise Asset Repair & Mitigation Management System (EARMMS)

Table of Contents

1. Introduction
2. Purpose of this Document
3. How to Use this Document
4. Project Overview
5. Business Context
6. Solution Overview
7. End-to-End User Journey
8. System Architecture
9. Standardized Data Contract (Common Reference)
10. Standardized API Contract (Common Reference)
11. Functional Requirements by Module
12. Technology-Wise Scope & Responsibilities
 - 12.1 SAP CAP
 - 12.2 SAPUI5
 - 12.3 SAP Build Process Automation (BPA)
 - 12.4 SAP CPI
 - 12.5 SAP AI (GenAI Hub)
 - 12.6 SAP Analytics
13. Inter-Team Contracts & Mock Strategy
14. 3-Week Timeline & Milestones
15. Development Standards
16. Acceptance Criteria
17. Glossary
18. Appendix

1. Introduction

The **Enterprise Asset Repair & Mitigation Management System (EARMMS)** is a SAP BTP-based solution that digitizes and automates the lifecycle of enterprise asset repair and urgent asset mitigation (temporary replacement). It is designed as a lightweight, SAP-native alternative to heavy ITSM tools, with a unique dual-flow model: **Repair Request (RR)** and **Mitigation Request (MR)**.

2. Purpose of this Document

This document serves as:

- The **single source of truth** for the complete EARMMS requirement
- A **standardized reference** for all developers across all skill streams
- A **guiding light** that every resource will follow to avoid inconsistency
- A **contract-based blueprint** enabling parallel, independent development

All resources are expected to **first read the full document** to understand the end-to-end scope, then focus on their **specific skill section** to understand their individual responsibilities.

3. How to Use this Document

Step	Action
Step 1	Read Sections 1 through 10 to understand the full solution and standards
Step 2	Navigate to Section 12 and find the sub-section for your assigned skill
Step 3	Review Section 13 (Inter-Team Contracts) to understand what you will mock and what others will provide
Step 4	Follow Section 14 for the timeline and your milestones
Step 5	Adhere to Section 15 (Development Standards) across all streams

4. Project Overview

Attribute	Detail
Project Name	EARMMS
Phase	Phase 1 (MVP)
Duration	2+1 Weeks
Target Users	Employees, Technicians, IT Admins
Platform	SAP BTP
Deployment	Cloud Foundry

5. Business Context

5.1 Problem Statement

Enterprises struggle with manual, fragmented asset repair processes. When assets break, employees email IT, wait days, and urgent cases have no structured fast-track. There is no visibility into spare inventory, SLA compliance, or asset health.

5.2 Business Goals

- Reduce asset downtime by 40%
- Achieve 90%+ SLA compliance
- Provide < 4-hour fulfillment for urgent mitigation
- Enable real-time visibility for all stakeholders

5.3 Target Outcome

A **working end-to-end MVP** in 3 weeks that demonstrates the full Repair + Mitigation lifecycle with automation, intelligence, integration, and analytics.

6. Solution Overview

EARMMS provides **two core flows**:

Repair Request (RR) Flow

Employee raises a request → System auto-classifies issue using AI → BPA auto-assigns technician → Technician resolves → Employee confirms → Ticket closes.

Mitigation Request (MR) Flow

Employee escalates to mitigation with business justification → BPA routes for Manager + IT Admin approval → Spare asset allocated from pool → Replacement issued → Original asset repaired → Replacement returned → Case closed.

Supporting capabilities: SLA monitoring, email notifications, real-time analytics dashboard.

7. End-to-End User Journey

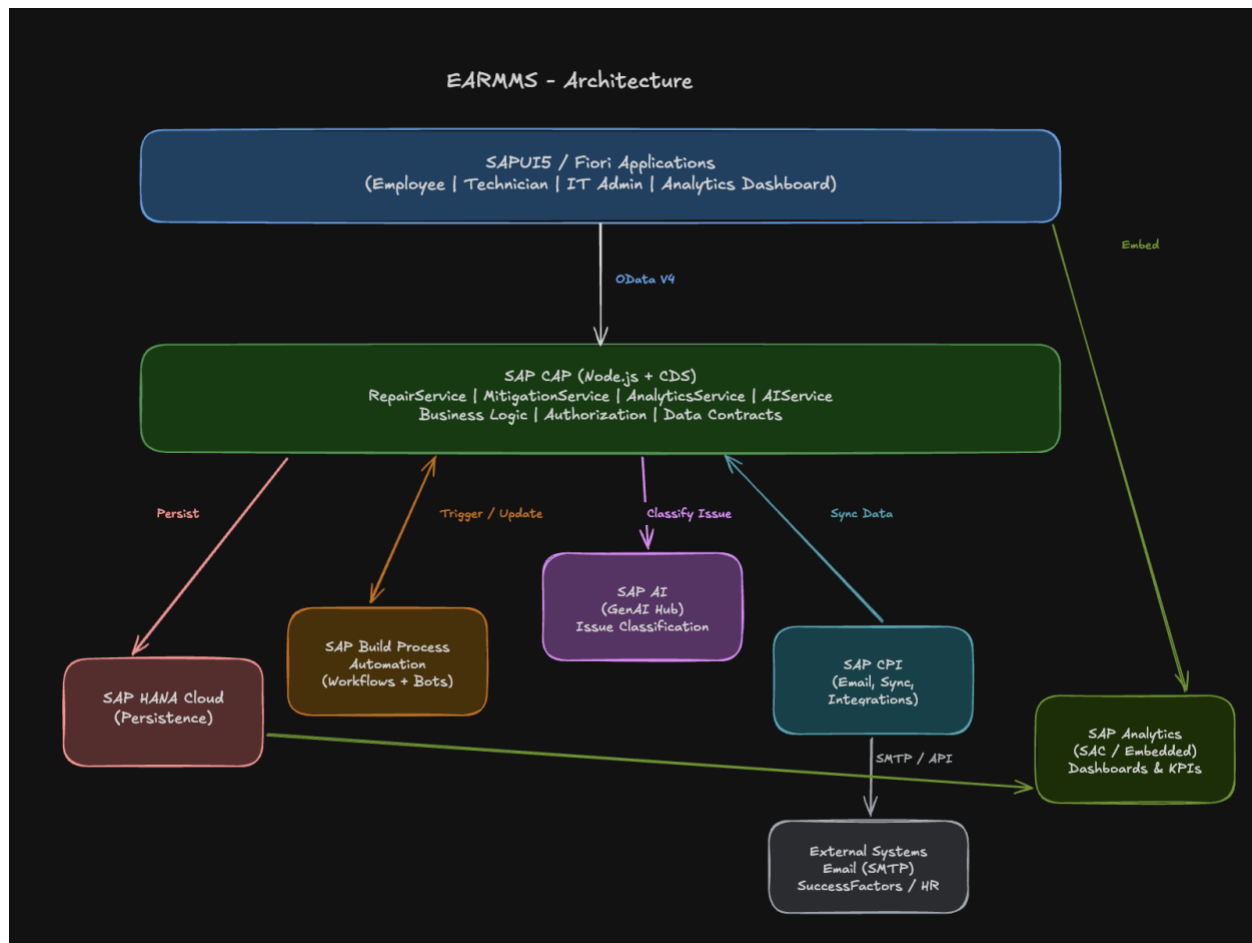
[Employee] → Opens Employee App
→ Views "My Assets"
→ Clicks "Raise Repair Request"
→ Fills form (asset, issue description, severity, photo)
→ Submits
↓
[System] → Generates RR-2026-00001
→ Calls AI service → auto-categorizes as "Hardware"
→ Persists in HANA via CAP service

↓
[BPA] → Workflow triggered
→ Finds technician with specialization + lowest load
→ Assigns ticket
→ Triggers email notification via CPI
↓
[Technician] → Opens Technician App
→ Sees ticket in queue (SLA-sorted)
→ Starts work → updates status → resolves
↓
[Employee] → Receives notification → confirms resolution
↓
[System] → Ticket closed → KPIs updated in analytics

— OR, if URGENT —

[Employee] → From open RR, clicks "Raise Mitigation"
→ Provides urgency reason + business impact
↓
[BPA] → Routes to Manager → IT Admin for approval
↓
[IT Admin] → Reviews → allocates spare from pool → approves
↓
[System] → Reserves spare → notifies employee
↓
[Employee] → Picks up replacement → works uninterrupted
→ Later returns replacement when original is repaired
↓
[System] → Updates spare pool → closes MR

8. System Architecture



9. Standardized Data Contract (Common Reference)

⚠ All developers MUST use these entity definitions and field names. No deviations.

9.1 Core Entities

Asset

Field	Type	Notes
assetId	UUID	Primary key
assetTag	String(20)	e.g., LAP-2026-0451
assetType_typeId	String(20)	FK → AssetType
make	String(50)	
model	String(50)	
serialNumber	String(50)	
purchaseDate	Date	
warrantyExpiry	Date	

currentOwner_employeeId	String(20)	FK → Employee
location	String(100)	
status	String(20)	Active, UnderRepair, InMitigation, Retired

AssetType

Field	Type	Notes
typeId	String(20)	PK (LAPTOP, MONITOR, CHAIR)
typeName	String(100)	
category	String(20)	IT, FURNITURE, ELECTRONICS
isCritical	Boolean	
standardSLAHours	Integer	

Employee

Field	Type	Notes
employeeId	String(20)	PK
name	String(100)	
email	String(100)	
department	String(50)	
designation	String(50)	
manager_employeeId	String(20)	FK (self)
isVIP	Boolean	

Technician

Field	Type	Notes
technicianId	UUID	PK
name	String(100)	
email	String(100)	
specialization	String(50)	Laptop, Networking, Furniture
currentLoad	Integer	Active ticket count

RepairRequest

Field	Type	Notes
-------	------	-------

requestId	UUID	PK
requestNumber	String(20)	RR-YYYY-NNNNN
asset_assetId	UUID	FK
raisedBy_employeeId	String(20)	FK
raisedOn	Timestamp	
issueCategory	String(50)	AI-populated
issueDescription	String(500)	
severity	String(10)	Low, Medium, High
photoUrl	String(500)	
status	String(20)	Open, Assigned, InProgress, Resolved, Closed
assignedTechnician_technicianId	UUID	FK
expectedResolution	Timestamp	
actualResolution	Timestamp	
resolutionNotes	String(1000)	
slaStatus	String(15)	OnTrack, AtRisk, Breached

MitigationRequest

Field	Type	Notes
mitigationId	UUID	PK
mitigationNumber	String(20)	MR-YYYY-NNNNN
linkedRepair_requestId	UUID	FK
raisedBy_employeeId	String(20)	FK
urgencyReason	String(500)	
businessImpact	String(20)	Critical, High, Medium
approvalStatus	String(20)	Pending, Approved, Rejected
approver_employeeId	String(20)	FK
replacementAsset_assetId	UUID	FK
issuedOn	Timestamp	
returnDueDate	Date	
returnedOn	Timestamp	
status	String(20)	Requested, Approved, Issued, Returned, Closed

SpareAssetPool

Field	Type	Notes
poolId	UUID	PK
asset_assetId	UUID	FK
availabilityStatus	String(20)	Available, Reserved, InUse
reservedFor_mitigationId	UUID	FK

SLAConfiguration

Field	Type	Notes
configId	UUID	PK
assetType_typeId	String(20)	FK
severity	String(10)	
responseTimeHours	Integer	
resolutionTimeHours	Integer	
mitigationSLAHours	Integer	

10. Standardized API Contract (Common Reference)

 All UI, BPA, CPI, and AI integrations **MUST** use these exact endpoints.

Base URL: /odata/v4/earmms

10.1 Repair Service

Endpoint	Method	Purpose
/RepairRequest	GET	List all RRs (filtered by user role)
/RepairRequest(id)	GET	Get single RR
/RepairRequest	POST	Create new RR
/RepairService.raiseRR	ACTION	Create RR with AI classification

/RepairService.assignTechnician	ACTION	Assign a technician
/RepairService.updateStatus	ACTION	Update ticket status
/RepairService.resolveRR	ACTION	Mark as resolved
/RepairService.closeRR	ACTION	Close the ticket

10.2 Mitigation Service

Endpoint	Method	Purpose
/MitigationRequest	GET	List MRs
/MitigationService.raiseMR	ACTION	Create MR linked to RR
/MitigationService.approveMR	ACTION	Approve mitigation
/MitigationService.rejectMR	ACTION	Reject mitigation
/MitigationService.issueReplacement	ACTION	Issue replacement asset
/MitigationService.returnReplacement	ACTION	Return replacement

10.3 Master Data

Endpoint	Method	Purpose
/Asset	GET	List assets
/Employee	GET	List employees
/Technician	GET	List technicians
/SpareAssetPool	GET	List spare assets

10.4 Analytics Service

Endpoint	Method	Purpose
----------	--------	---------

/AnalyticsService.getKPIs	FUNCTION	Returns aggregated KPIs
/AnalyticsService.getTrends	FUNCTION	Returns time-series data

10.5 AI Service (CAP Wrapper)

Endpoint	Method	Purpose
/AIService.classifyIssue	ACTION	Input: description text; Output: category

11. Functional Requirements by Module

11.1 Authentication & Authorization

- Users authenticate via XSUAA / SSO
- Role-based access: Employee, Technician, IT Admin
- Employees see only their own data; Technicians see assigned tickets; IT Admin sees all

11.2 Asset Management

- View list of all assets (IT Admin)
- View assigned assets (Employee)
- Asset status auto-updates based on RR/MR lifecycle

11.3 Repair Request Management

- Employee raises RR with asset, description, severity, photo
- AI auto-classifies issue category
- BPA auto-assigns technician
- Technician updates status and resolves
- Employee confirms, ticket closes
- SLA timer calculates based on configuration

11.4 Mitigation Request Management

- Employee raises MR linked to open RR
- BPA routes for Manager → IT Admin approval
- IT Admin allocates spare
- Replacement issued, tracked, returned

11.5 Spare Pool Management

- IT Admin views available spares
- Spare status auto-updates on allocation / return

11.6 Notifications

- Email via CPI on key events: RR created, assigned, resolved; MR approved, issued; SLA at-risk

11.7 Analytics

- Dashboard with KPIs: Open Tickets, SLA Compliance %, MTTR, Mitigation Count
- Charts: tickets by status, MTTR trend, tickets by asset type

11.8 SLA Monitoring

- Bot runs every 15 minutes
- Flags ticket as OnTrack / AtRisk / Breached
- Sends escalation email on breach

12. Technology-Wise Scope & Responsibilities

12.1 SAP CAP Developer

Role: Build the backbone data model, services, and business logic. You are the **foundation provider** for the entire application.

Scope

ID	Task
CAP-01	Set up CAP project structure with db/, srv/, app/ folders
CAP-02	Define all 8 entities in db/schema.cds per Section 9
CAP-03	Deploy schema to SAP HANA Cloud
CAP-04	Implement RepairService.cds and handler logic
CAP-05	Implement MitigationService.cds and handler logic

CAP-06	Implement <code>AnalyticsService.cds</code> with aggregation queries
CAP-07	Implement <code>AIService.cds</code> wrapper that internally calls GenAI endpoint
CAP-08	Auto-generate RR/MR sequence numbers
CAP-09	Calculate SLA due dates based on severity + asset type
CAP-10	Validate asset ownership and business rules
CAP-11	Configure XSUAA & role-based <code>@restrict</code> annotations
CAP-12	Seed mock master data (CSV files in <code>db/data/</code>)
CAP-13	Expose OData V4 endpoints for all services
CAP-14	Publish API documentation (Postman collection) by end of Week 1

Independence Strategy

- Build and deploy services using **mock data** in CSV files
- Do NOT wait for UI, BPA, or AI — expose all endpoints with working logic

- Provide **stub implementation** for `AIService.classifyIssue` returning hardcoded “Hardware” until AI team delivers
- Publish Postman collection on **Day 5** so other teams can consume your APIs

Deliverables

- Deployed CAP service on BTP
- Postman collection with all endpoints
- Seeded master data (10 assets, 5 employees, 3 technicians, 5 spare assets)
- HANA schema live

12.2 SAPUI5 Developer

Role: Build user-facing applications for all personas. You are the **face of the application**.

Scope

ID	Task
UI-01	Employee App: My Assets, Raise RR (with photo upload), Raise MR, My Requests tracker
UI-02	Technician App: My Queue (SLA-sorted), Ticket detail, Update status, Resolve
UI-03	IT Admin App: Mitigation Approval Inbox, Spare Pool View, Technician Assignment
UI-04	Analytics Dashboard App: KPI tiles + 3 charts (embed SAC or use UI5 charts)
UI-05	Implement Fiori Launchpad with role-based tile navigation
UI-06	Use Fiori Elements for list-detail pages; freestyle UI5 for dashboards/wizards
UI-07	Responsive design (desktop + tablet + mobile)
UI-08	Status color coding (Green/Amber/Red)
UI-09	Photo upload component for RR
UI-10	Integrate with CAP OData services per Section 10

Independence Strategy

- Use the **Mock Server feature** of UI5 tooling
- Create mock JSON responses that match the data contract in Section 9
- Begin UI development on **Day 1** without waiting for CAP backend
- Switch from mock server to real CAP endpoint on Day 8 after integration

Deliverables

- 4 working UI5 apps
 - Deployed on Fiori Launchpad (SAP Build Work Zone)
 - Mock-to-real backend switch verified
-

12.3 SAP Build Process Automation (BPA) Developer

Role: Build workflows and bots that automate the business processes. You are the **automation engine**.

Scope

ID	Task
BPA-01	Repair Request Workflow: Trigger on RR creation → fetch technicians → select lowest-load → assign via CAP API → trigger email
BPA-02	Mitigation Approval Workflow: On MR creation → route to Manager → if approved, route to IT Admin → on approval, call issueReplacement API
BPA-03	SLA Monitor Bot: Scheduled every 15 minutes → fetch all open RRs → calculate SLA % → update status (OnTrack/AtRisk/Breached) → trigger escalation email for breaches
BPA-04	Return Reminder Bot: Daily schedule → check MRs with returnDueDate within 2 days → send reminder email
BPA-05	Approval forms for Manager and IT Admin (mobile-friendly)
BPA-06	Integration with CAP APIs via HTTP action
BPA-07	Integration with CPI iFlow for email notifications

Independence Strategy

- Use **mock HTTP endpoints** (e.g., Postman Mock Server or SAP API Business Hub sandbox) simulating CAP APIs
- Build workflows with dummy payloads matching Section 9 data contract
- Switch to real CAP endpoints on Day 10 after CAP APIs are published

Deliverables

- 2 deployed workflows
 - 2 scheduled bots
 - Approval forms working
 - Test cases with sample payloads
-

12.4 SAP CPI Developer

Role: Build integration flows (iFlows) that connect EARMMS with external services. You are the **integration layer**.

Scope

ID	Task
CPI-01	iFlow: Email Notification — Receive payload from BPA → format email template → send via SMTP
CPI-02	iFlow: Mock Employee Sync — Scheduled iFlow that reads mock CSV / REST endpoint → transforms → writes to CAP Employee entity via OData
CPI-03	Implement error handling and retry logic
CPI-04	Parameterize email templates (RR created, MR approved, SLA breach, return reminder)
CPI-05	Secure endpoints with OAuth 2.0

Independence Strategy

- Build iFlows using **mock inbound payloads** matching the standard contract
- Use SAP CPI's built-in testing tool to simulate triggers
- Connect to real CAP endpoint on Day 10 for live sync; until then, write to a test endpoint

Deliverables

- 2 deployed iFlows
 - Email templates for 4 scenarios
 - Integration test log
-

12.5 SAP AI Developer (GenAI Hub)

Role: Add intelligence to the application via AI-powered issue classification. You are the **intelligence layer**.

Scope

ID	Task
AI-01	Provision SAP GenAI Hub access and select LLM (e.g., GPT-4, Claude)
AI-02	Design prompt for issue classification (Hardware / Software / Physical Damage / Other)
AI-03	Implement REST endpoint that accepts text and returns category + confidence
AI-04	Handle error cases (empty text, API failure) with fallback to "Other"
AI-05	Provide API specification document for CAP team to consume
AI-06	Test with 20+ sample descriptions for accuracy

Independence Strategy

- Develop AI endpoint as a **standalone REST service** (Node.js or Python Cloud Foundry app)
- Publish API contract on Day 3 so CAP team can integrate via stub
- Replace stub with real endpoint on Day 12

API Contract (Published)

POST /ai/classify

Request: { "description": "My laptop screen is cracked" }

Response: { "category": "Physical Damage", "confidence": 0.92 }

Deliverables

- Deployed GenAI service endpoint
- API documentation
- Accuracy test report

12.6 SAP Analytics Developer

Role: Build the analytics dashboard that provides insights to leadership. You are the **insights provider**.

Scope

ID	Task
AN-01	Connect SAC to HANA Cloud (or build embedded analytics in UI5 if SAC unavailable)
AN-02	Build dataset models for Repair and Mitigation

AN-03	KPI Tiles: Open Tickets, SLA Compliance %, MTTR, Mitigation Count
AN-04	Charts: Tickets by status (pie), MTTR trend (line), Tickets by asset type (bar)
AN-05	Implement drill-down from KPI → ticket list
AN-06	Embed SAC story in UI5 Analytics App via iFrame or SAC SDK
AN-07	Verify dashboard refreshes on real-time data

Independence Strategy

- Build dashboard using **mock HANA views or CSV imports** matching final schema
- Publish KPI definitions on Day 3 so CAP team can expose correct aggregations
- Switch to live HANA connection on Day 12

Deliverables

- SAC story / embedded dashboard
- 4 KPI tiles + 3 charts
- Drill-down functionality
- Embedded view in UI5 app

13. Inter-Team Contracts & Mock Strategy

Principle: Every team starts Day 1 using mocks. Integration happens in Week 3.

Key Rule: All teams publish their **API specs by Day 3**, which become the contracts.

14. 3-Week Timeline & Milestones

Provider → Consumer	Contract	Mock Strategy	Integration Day
CAP → UI5	OData services (Section 10)	UI5 mock server with JSON	Day 8
CAP → BPA	REST/OData actions	Postman mock server	Day 10
AI → CAP	/ai/classify REST	Stub in CAP returning hardcoded value	Day 12
CAP → Analytics	HANA views / OData aggregations	CSV import	Day 12
BPA → CPI	HTTP trigger with payload	Mock inbound in CPI	Day 12
CPI → External	SMTP / REST	Test SMTP server	Day 12

Week 1 — Foundation & Parallel Kickoff (Day 1 to 7)

Day	CAP	UI5	BPA	CPI	AI	Analytics
1	Project setup + schema draft	Project setup + mock server	Workflow design	iFlow design	GenAI Hub setup	SAC setup
2	Entity definitions	Employee App layout	Process modeling	Email template design	Prompt design	Dataset modeling
3	Publish data contract	Mock data creation	Publish workflow specs	Publish iFlow specs	Publish AI API spec	Publish KPI spec
4	HANA deployment	Raise RR form	Workflow build starts	Email iFlow build	Endpoint build	Tile build
5	Publish Postman collection	Raise MR form	Approval forms	Testing with mocks	Testing prompts	Chart build
6	Service logic (RR)	Track Requests page	Workflow testing	Mock sync iFlow	API hosting	Drill-down
7	Service logic (MR)	Technician App	SLA bot build	iFlow testing	API documentation	Dashboard ready

Milestone: All teams have mock-based working deliverables

Week 2 — Core Build & Module Completion (Day 8 to 14)

Day	CAP	UI5	BPA	CPI	AI	Analytics
8	Auth + validation	Switch to real CAP endpoint	Return reminder bot	Error handling	Testing accuracy	KPI refinement
9	Analytics service	IT Admin App	Workflow refinement	Retry logic	Edge cases	Embed in UI5 prep
10	Integration fixes	IT Admin App (continued)	Integrate with real CAP	Integrate with real CAP	Deployment	Dashboard polish
11	Performance tuning	Analytics App	Approval flows	Live email testing	Performance	Live data test
12	Full API freeze	Dashboard embedding	Integrate with CPI	Bot integration	CAP integration	Live data connect
13	Bug fixes	Bug fixes	Bug fixes	Bug fixes	Bug fixes	Bug fixes
14	Documentation	Documentation	Documentation	Documentation	Documentation	Documentation

Milestone: Each module working end-to-end on real data (no mocks)

Week 3 — End-to-End Integration, Testing & Demo (Day 15 to 21)

Day	Focus	Activities
Day 15	End-to-End Integration — Repair Flow	• Full RR flow: Employee raises RR → AI classifies → BPA auto-assigns technician → CPI email fires → Technician resolves → Employee confirms → Ticket closes
		• SLA monitor bot verification
		• Log all integration issues
Day 16	End-to-End Integration — Mitigation Flow + Analytics	• Full MR flow: Employee raises MR → BPA routes to Manager → IT Admin approval → Spare allocation → Replacement issued → Return → Spare pool updated data and verified
		• Return reminder bot verification
		• Analytics dashboard connected to live
Day 17	Bug Fixes, UI Polish & Demo Prep	• Fix all critical/major defects
		• UI polish (colors, spacing, error messages)
		• Demo data seeding (create realistic scenarios)
		• Performance check
		• Demo rehearsal
		• Finalize documentation
Day 18	Stakeholder Demo	• Live demo to stakeholders

Milestone: Production-ready MVP + successful stakeholder demo

15. Development Standards

15.1 Coding Standards

- **CAP:** Follow SAP CAP best practices, use CDS linter
- **UI5:** Follow SAP Fiori Design Guidelines, ESLint config
- **BPA:** Use descriptive workflow names; version every workflow
- **CPI:** Use camelCase for variables; comment every step
- **AI:** Document prompts and model version
- **Analytics:** Use consistent measure names

15.2 Naming Conventions

- Entities: PascalCase (RepairRequest)
- Fields: camelCase (raisedBy, issueCategory)
- Services: PascalCase ending with Service (RepairService)
- Workflows: WF_EARMMS_<Purpose> (WF_EARMMS_RepairLifecycle)

- iFlows: IF_EARMMS_<Purpose> (IF_EARMMS_EmailNotification)

15.3 Version Control

- Single Git repository with branch per skill stream
- Daily commits

15.4 Documentation

- Each developer maintains a module README
- Postman collections for all APIs
- Screenshots of key screens / workflows

15.5 Testing

- Unit tests for CAP service handlers
 - UI5 end-to-end testing
 - BPA workflow test cases with sample payloads
 - CPI iFlow testing with mock payloads
-

16. Acceptance Criteria

The Phase 1 MVP is accepted when:

- Employee can raise an RR, AI classifies the issue, and BPA auto-assigns technician
 - Email notification is sent on RR creation and status changes
 - Technician can view queue (SLA-sorted), update and resolve tickets
 - Employee can raise MR; BPA routes through Manager and IT Admin approval
 - Spare asset is allocated and replacement is issued
 - Replacement return flow works; spare returns to pool
 - SLA monitor bot updates SLA status and escalates on breach
 - Analytics dashboard shows live KPIs and charts
 - CPI email iFlow delivers to configured recipients
 - End-to-end demo (Section 14 Day 18) runs without critical defects
-

17. Glossary

Term	Definition
EARMMS	Enterprise Asset Repair & Mitigation Management System
RR	Repair Request
MR	Mitigation Request

SLA	Service Level Agreement
MTTR	Mean Time To Repair
VIP	Very Important Person (auto-approval flag)
CAP	SAP Cloud Application Programming Model
BPA	SAP Build Process Automation
CPI	SAP Cloud Platform Integration
SAC	SAP Analytics Cloud
BTP	SAP Business Technology Platform
iFlow	Integration Flow in SAP CPI

18. Appendix

18.1 Reference Documents

- CAP Documentation: <https://cap.cloud.sap>
- SAPUI5 Documentation: <https://ui5.sap.com>
- BPA Documentation: <https://help.sap.com/docs/build-process-automation>
- CPI Documentation: <https://help.sap.com/docs/cloud-integration>
- GenAI Hub Documentation: <https://help.sap.com/docs/sap-ai-core>
- SAC Documentation: https://help.sap.com/docs/SAP_ANALYTICS_CLOUD

18.2 Contact Points

Role	Name	Responsibility
UI5 Developer	Jyoti Ranjan Rout, Manish Singh	UI5 apps
BPA Developer	Sushree Sangita Das, Chantu (BUCHUPALLI REDDY)	Workflows & bots
CPI Developer	Vagesh Chattaraki, PRANJAL PANDEY	Integrations
AI Developer	Milan, PRANJAL PANDEY	GenAI classification

Analytics Developer	Milan, Chantu (BUCHUPALLI REDDY)	Dashboard
CAP Developer	Hemanth Reddy Vanukuri, Jyoti Ranjan Rout (Support)	Services

End of Document — EARMMS Functional Requirements
